



A three-scene game

Overview

Students build a complete (tiny) game with a beginning, a middle and an end. The Phaser ideas are **scenes and the scene manager**, **passing data between scenes**, and **pointer input on game objects**; the TypeScript ideas are **interfaces**, **object literals**, **union types** and the **spread operator**. The one conceptual trap - scene objects are reused, so fields keep old values - is worth a whole discussion: it is the most common "my game works once" bug students will write this term.

Prerequisites

- Chapter 1: building and serving, config/scene/game object, `preUpdate` and `delta`
- Java interfaces (for the analogy with TypeScript interfaces)

Learning outcomes

Students can:

1. structure a game as several scenes, and move between them with `this.scene.start(key, data)`
2. pass data between scenes through `init(data)`, described by an `interface`
3. explain why a restarted scene keeps its field values, and reset them in `init()`
4. make a game object respond to the pointer with `setInteractive()` and `"pointerdown"`, and detect clicks on nothing with `this.input.on("pointerdown", ...)`
5. choose between `on` and `once` for an event
6. share a value between scenes with `this.registry`

Suggested session plan (2 hours)

Time	Activity
0:00 - 0:10	Play <code>ch02_click_the_ball</code> together. Ask: "how many screens? what moves you between them?" Draw the scene-flow diagram on the board before showing it
0:10 - 0:30	Slides 3-8: scenes, keys, <code>scene.start</code> , <code>on</code> vs <code>once</code> , loading once. Walk through <code>StartScene.ts</code>
0:30 - 0:45	Slides 9-11: <code>setInteractive</code> , <code>pointerdown</code> , the clock. Walk through <code>PlayScene.ts</code>
0:45 - 1:00	Slides 12-13: passing data, interfaces. Challenges 1-3
1:00 - 1:15	Live demo of the trap: in the plus version, comment out the body of <code>init()</code> in <code>PlayScene</code> , win once, play again - the first click wins. Ask students to explain before revealing slide 14
1:15 - 1:30	Slides 15-17: the plus version: misses, encapsulation, sound, registry, spread
1:30 - 2:00	Challenges 4-6

Key points to stress

- `scene.start` **shuts down** the current scene: its game objects are destroyed and its listeners removed. Students sometimes expect the old scene to be "underneath" still
- scenes talk to each other through **data passed to** `start` (one-off hand-over) or the **registry** (shared, long-lived). Resist letting students reach into other scenes with `this.scene.get(...)` to read their fields - it couples scenes tightly (Chapter 6 shows events, which are better still)
- the object is reused: `init()` **is where a scene resets itself**
- the `Ball` class did not change to become clickable - the **scene** decided. A good moment to talk about keeping classes focused

Common problems and errors

What students see	Cause	Fix
Pressing SPACE does nothing; no error	scene key misspelled as a string ("Playscene")	use the constants in <code>keys.ts</code>
Second round starts with the old score/hits, or ends instantly	fields reset only where declared	reset in <code>init()</code>
Property 'seconds' does not exist on type 'object'	<code>init(data: object)</code> or untyped <code>data</code>	type the parameter with the interface
Object literal may only specify known properties, and 'secs' does not exist in type 'WinData'	misspelled property in the data	this is the interface doing its job
Clicking the ball does nothing	forgot <code>setInteractive()</code>	game objects ignore the pointer until made interactive
Every click counts as a miss, including hits	the scene-wide listener ignores the <code>over</code> parameter	check <code>over.length === 0</code>
No sound in the plus version	the browser blocks audio until the page has been clicked or a key pressed	click the game first; mention Chapter 5
'best' is possibly 'undefined'	using <code>best.toFixed()</code> without checking	this is the union type <code>`number`</code>

Discussion questions

- Why keep scene keys in one file? What other strings in a game deserve the same treatment? (Asset keys, event names, registry names.)
- `WinScene` needs the time. Name three ways it could get it. (Data passed to `start`, the registry, reading `PlayScene`'s field.) Which is best, and why?
- In the plus version the ball's speed is private and the scene calls `speedUp()`. What would be lost if the scene set `ball.speedX` directly?

- Is "Too slow!" (challenge 4) a win scene any more? When is it right to reuse a scene with a flag, and when should it be a new scene?
-

Extension ideas

- add a pause: `this.scene.pause()` and `this.scene.resume()` , and a "PAUSED" text
 - launch a second scene **on top** of the play scene (`this.scene.launch(...)`) for a HUD - a preview of Chapter 6
 - make the ball a different random colour each round with `setTint(...)`
-

Assessment ideas

- Practical: "add a fourth scene - an instructions screen between start and play - with its own key, reached by SPACE, left by SPACE"
 - Code reading: show `PlayScene` without `init()` , and a transcript of two rounds; ask for the bug and the fix
 - Short answer: the difference between `this.registry` and the data passed to `this.scene.start`
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. Restyle

Project: `solutions/ch02_challenge_1_restyle` (from `ch02_click_the_ball`)

Goal: find where each thing is decided - the picture, the speed, the words - and change it.

`src/objects/Ball.ts`

```
export const BALL_KEY = "ball";  
export const BALL_FILE = "assets/images/ball_blue.png";
```

`src/scenes/PlayScene.ts`

```
const BALL_SPEED = 520;
```

plus new wording and colours in `StartScene.create()` and `WinScene.create()`, and `ball_blue.png` copied from `assets/images/` into the project's `public/assets/images/`.

Look for: the file copied into `public/` (the most common failure: the path is right but the file is not there, and Phaser shows a green-and-black "missing texture" square). Changing `BALL_FILE` but keeping the key `"ball"` is fine - the key is just a name.

2. Enter works too

Project: `solutions/ch02_challenge_2_enter_works_too` (from `ch02_click_the_ball`)

Goal: two listeners, one action - and no duplicated code.

src/scenes/StartScene.ts (and the same pattern in WinScene.ts)

```
this.input.keyboard!.once("keydown-SPACE", () => {
  this.next();
});
this.input.keyboard!.once("keydown-ENTER", () => {
  this.next();
});
}

private next(): void {
  this.scene.start(PLAY_SCENE);
}
```

Look for: the action in one method rather than written twice; the on-screen messages updated to match. Some students look for a single event for "SPACE or ENTER" - there is not one, and two listeners is the normal answer. A student who uses `this.input.keyboard!.on("keydown", (event) => ...)` and checks `event.code` has found a valid alternative.

3. Give up

Project: `solutions/ch02_challenge_3_give_up` (from `ch02_click_the_ball`)

Goal: a new way out of a scene, and confidence that restarting a scene rebuilds it.

src/scenes/PlayScene.ts

```
this.input.keyboard!.once("keydown-ESC", () => {
  this.scene.start(START_SCENE);
});
```

The start scene "works exactly as it did the first time" because its `create()` runs again, making fresh text and a fresh SPACE listener. Nothing needs resetting: `StartScene` has no fields.

Look for: the import of `START_SCENE`. Tested with the harness: SPACE, ESC, SPACE goes Start -> Play -> Start -> Play.

4. Too slow!

Project: `solutions/ch02_challenge_4_too_slow` (from `ch02_click_the_ball`)

Goal: a timer event, and a scene that shows two different outcomes from its data.

`src/scenes/PlayScene.ts`

```
const TIME_LIMIT = 5000;
...
this.time.delayedCall(TIME_LIMIT, () => {
  this.lose();
});
...
private win(): void {
  const seconds = (this.time.now - this.startTime) / 1000;
  const data: WinData = { won: true, seconds: seconds };
  this.scene.start(WIN_SCENE, data);
}

private lose(): void {
  const data: WinData = { won: false, seconds: TIME_LIMIT / 1000 };
  this.scene.start(WIN_SCENE, data);
}
```

`src/scenes/WinScene.ts`

```
export interface WinData {
  won: boolean;
  seconds: number;
}
...
const heading = this.won ? "You got it!" : "Too slow!";
const detail = this.won ? `in ${this.seconds.toFixed(2)} seconds` : `You had
${this.seconds} seconds`;
```

The timer belongs to the play scene's clock, so when the player wins and the scene shuts down, the timer is thrown away with it - it can never fire after a win. Worth asking students *why* it does not.

Look for: adding `won` to the interface - the compiler then insists both `win()` and `lose()` say which it is. Checking elapsed time in `update()` instead of `delayedCall` also works; accept it, but point out the timer is simpler and cannot be forgotten.

5. The decoy

Project: `solutions/ch02_challenge_5_the_decoy` (from `ch02_click_the_ball_plus`)

Goal: reuse a class with a parameter; understand how the "missed everything" listener sees the world.

`src/objects/Ball.ts`

```
export const DECOY_KEY = "decoy";
export const DECOY_FILE = "assets/images/ball_blue.png";
...
constructor(scene: Phaser.Scene, x: number, y: number, speedX: number,
speedY: number, key: string = BALL_KEY) {
    super(scene, x, y, key);
```

`src/scenes/PlayScene.ts`

```
const decoy = new Ball(this, Phaser.Math.Between(100, 700),
Phaser.Math.Between(100, 500),
    Math.cos(decoyAngle) * DECOY_SPEED, Math.sin(decoyAngle) * DECOY_SPEED,
DECOY_KEY);
decoy.setInteractive({ useHandCursor: true });
decoy.on("pointerdown", () => {
    for (let i = 0; i < DECOY_MISSES; i++) {
        this.miss();
    }
});
```

The decoy is interactive, so a click on it puts it in the `over` list - the scene-wide listener does **not** count it as a miss, and the decoy's own listener counts two. Tested with the harness: decoy, ball, empty space gives 1 hit and 3 misses.

Look for:

- a **default parameter** (`key: string = BALL_KEY`) so the existing call still works - or a new subclass `Decoy` extends `Ball` , which is also a good answer
- the blue picture loaded in `StartScene`
- a common wrong answer: making the decoy *not* interactive and relying on the "missed everything" listener - that counts it as **one** miss, not two

6. Top five

Project: `solutions/ch02_challenge_6_top_five` (from `ch02_click_the_ball_plus`)

Goal: keep a structured value (an array) in the registry; basic array work.

`src/scenes/WinScene.ts`

```
const times: number[] = this.registry.get(BEST_TIMES) ?? [];
times.push(total);
times.sort((a, b) => a - b);
const top = times.slice(0, TABLE_SIZE);
this.registry.set(BEST_TIMES, top);

const place = top.indexOf(total) + 1;
```

`src/scenes/StartScene.ts`

```
const times: number[] = this.registry.get(BEST_TIMES) ?? [];
let table = "BEST TIMES\n";
if (times.length === 0) {
  table = table + "none yet";
}
times.forEach((time, index) => {
  table = table + `${index + 1}.  ${time.toFixed(2)} s\n`;
});
```

- `??` is the **nullish coalescing** operator: "this, unless it is `null` or `undefined` , in which case that"
- `sort()` on its own sorts numbers **as text** (so 10 comes before 9) - hence the compare function. This catches out nearly everyone; a great discussion point

- `indexOf` gives `-1` when the value is not there, so `place` is `0` when the time missed the table

Look for: the compare function in `sort` ; `slice` so the table never grows past five; a sensible message when the time does not make the table. Sharp students notice `indexOf(total)` finds the *first* equal time if two totals tie exactly - fine to accept.

Slides

Chapter 2

A three-scene game

Start - play - win

Time: 2.3



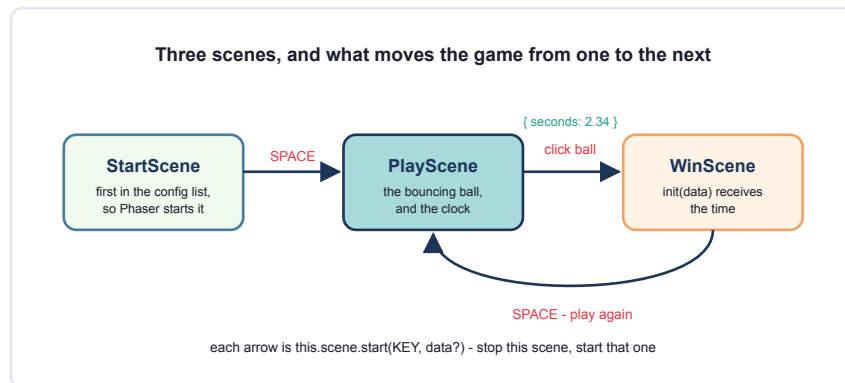
1 / 18

Today

- splitting a game into **scenes**
- moving between scenes; passing data along
- the trap: scenes are **reused**
- clicking on game objects
- `on` and `once`
- the **registry**: values every scene can see

2 / 18

Three scenes



```
scene: [StartScene, PlayScene, WinScene], // Phaser starts the FIRST
```

3 / 18

Scene keys - as constants

```
// keys.ts
export const START_SCENE = "StartScene";
export const PLAY_SCENE = "PlayScene";
export const WIN_SCENE = "WinScene";

// PlayScene.ts
constructor() {
  super(PLAY_SCENE);
}
```

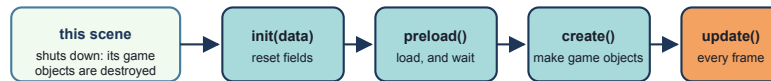
`start("Playscene")` fails **silently**. `start(PLAYSCENE)` fails the **build**.

4 / 18

Starting a scene

```
this.input.keyboard!.once("keydown-SPACE", () => {  
  this.scene.start(PLAY_SCENE);  
});
```

What this.scene.start(PLAY_SCENE, data) does



The scene OBJECT is not new: Phaser made it once, when the game began, and starts it again.
Its fields keep their old values - which is why init() puts them back.

5 / 18

on or once?

- `on(event, fn)` - every time
- `once(event, fn)` - the first time, then stop listening
- a scene's listeners are removed when it shuts down
- use `once` when something should happen once: it says what you mean

6 / 18

Load once, use everywhere

```
// StartScene - the first scene
preload(): void {
    this.load.image(LOGO_KEY, LOGO_FILE);
    this.load.image(BALL_KEY, BALL_FILE);
}
```

Loaded files belong to the **game**. `PlayScene` has no `preload()` .

7 / 18

Clicking a game object

```
const ball = new Ball(this, 400, 300, speedX, speedY);

ball.setInteractive({ useHandCursor: true });

ball.on("pointerdown", () => {
    this.win();
});
```

- ignored by the pointer until `setInteractive()`
- **pointer** = mouse *or* finger
- `Ball` does not know it can be clicked - the scene decides

8 / 18

Timing

```
create(): void {  
    this.startTime = this.time.now;  
}  
  
override update(time: number, _delta: number): void {  
    const seconds = (time - this.startTime) / 1000;  
    this.timeText.setText(`Time: ${seconds.toFixed(1)}`);  
}
```

`this.time` - the scene's clock, in milliseconds

9 / 18

Try it: Click the Ball

- build, serve, play `ch02_click_the_ball`
- find the three `this.scene.start(...)` calls
- which scene loads the ball's picture? Which uses it?

PHASER 4

a guide for OO programmers

Click the bouncing ball as fast as you can

Press SPACE to start

10 / 18

Passing data

```
// PlayScene
const data: WinData = { seconds: seconds };
this.scene.start(WIN_SCENE, data);

// WinScene
export interface WinData {
  seconds: number;
}

init(data: WinData): void {
  this.seconds = data.seconds;
}
```

`{ seconds: 2.3 }` - an **object literal**. `interface` - its shape, checked.

11 / 18

The trap

```
private hits = 0; // runs ONCE - when the object is made
```

- Phaser makes **one** `PlayScene`, and starts the **same object** again
- game objects: new every time (`create()`)
- fields: **whatever the last round left**

```
init(): void { // runs EVERY time the scene starts
  this.hits = 0;
  this.misses = 0;
}
```

12 / 18

The plus version

- 5 hits to win; faster each time
- misses cost a second
- sounds
- a best time



13 / 18

Clicks on nothing

```
this.input.on("pointerdown",
  (_pointer: Phaser.Input.Pointer, over: Phaser.GameObjects.GameObject[]) => {
    if (over.length === 0) {
      this.miss();
    }
  });
```

`over` = the interactive game objects under the pointer

14 / 18

Ask, don't reach in

```
// Ball
public speedUp(factor: number): void {
    this.speedX = this.speedX * factor;
    this.speedY = this.speedY * factor;
}

// PlayScene
this.ball.speedUp(SPEED_UP);
this.ball.teleport();
```

15 / 18

The registry

```
// WinScene
const best: number | undefined = this.registry.get(BEST_TIME);
if (best === undefined || total < best) {
    this.registry.set(BEST_TIME, total);
}
```

- shared by **every scene**, for as long as the game runs
- `number | undefined` - a **union type**
- `{ ...style, fontSize: "64px" }` - **spread**: a copy, with changes

16 / 18

Summary

- `scene: [...]` - Phaser starts the first
- `this.scene.start(KEY, data)` -> the new scene's `init(data)`
- keys as constants; data described by an `interface`
- scenes are reused: reset in `init()`
- `setInteractive()` + `"pointerdown"` ; `this.input.on(...)` for every click
- `this.registry` - shared values

17 / 18

Challenges

1. **Restyle** - blue ball, faster, new words
2. **Enter works too** - ENTER does what SPACE does
3. **Give up** - ESC back to the title
4. **Too slow!** - 5 seconds, then "Too slow!" - no fourth scene
5. **The decoy** - a blue ball worth two misses
6. **Top five** - a best-times table in the registry

Next: **Chapter 3 - Preloading**

18 / 18